Assignment 2 — Software Quality III: Build and Release (CSCN73010)

Repository: https://github.com/crispyT-T/Flask_Image_Recognition

---

1. Project setup and deployment

What the project is

A Flask web application that classifies a photograph of a hand sign as a digit from 0 to 9, using a Keras model trained by transfer learning. The upstream project is Sachin-crypto/Flask_Image_Recognition.

Choosing a baseline

The upstream repository no longer holds only the legacy application. Between October and November 2024 it accumulated commits adding a CI workflow, a Pylint configuration, pre-commit hooks and six test files — the same assignment, already completed by someone else.

Forking the tip would therefore have presented another person's work as my own and left no room to demonstrate my own incremental history. This fork is instead based on `62d8114` ("Added dl model", 29 April 2023), the last commit before that work began. The history contains the original project's commits up to that point, then mine. The 2024 commits are not part of this branch.

Steps to set up and run

Requires Python 3.11.

```
git clone https://github.com/crispyT-T/Flask_Image_Recognition.git
cd Flask_Image_Recognition
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

The application serves on http://127.0.0.1:9000. Choosing an image from `test_images/` and submitting it returns the predicted digit.

Problems encountered, and how they were resolved

The pinned dependencies could not be installed. `requirements.txt` specified `tensorflow==2.10.0`. Installing it on Apple Silicon failed:

```
ERROR: Could not find a version that satisfies the requirement tensorflow==2.10.0
       (from versions: 2.13.0rc0, ... 2.22.0rc0)
ERROR: No matching distribution found for tensorflow==2.10.0
```

TensorFlow 2.10 predates the arm64 macOS wheels published under the `tensorflow` name, so no distribution exists for the platform; the oldest available is 2.13. The project was developed on Windows, where the pinned versions do install.

The resolution was TensorFlow **2.15.1** — the newest release that still bundles Keras 2. That constraint is not arbitrary: the saved model is a Keras 2 HDF5 file, and Keras 3, which ships with TensorFlow 2.16 and later, will not load that format without the `tf-keras` shim. NumPy was held at 1.26.4, the last 1.x release, because TensorFlow 2.15 is not built against NumPy 2. Werkzeug was held below 3.0 because Flask 2.2 imports symbols that Werkzeug 3 removed.

The model path was hardcoded to the original author's machine. `model.py` contained:

```
model = load_model("D:/SACHIN/Models/Hand-Sign-Digit-Language/digit_model.h5")
```

Importing the module therefore failed anywhere else:

```
OSError: No file or directory found at D:/SACHIN/Models/Hand-Sign-Digit-
Language/digit_model.h5
```

Since `digit_model.h5` is committed at the repository root, the path is now resolved relative to the module file, which works on any operating system and from any working directory:

```
MODEL_PATH = os.path.join(os.path.dirname(__file__), "digit_model.h5")
model = load_model(MODEL_PATH)
```

Worth noting: as published, the legacy application could not run anywhere except the author's own laptop.

Editor and cache files were tracked. `.idea/` and `__pycache__/` were committed. Both were removed and a `.gitignore` added.

Evidence that it runs

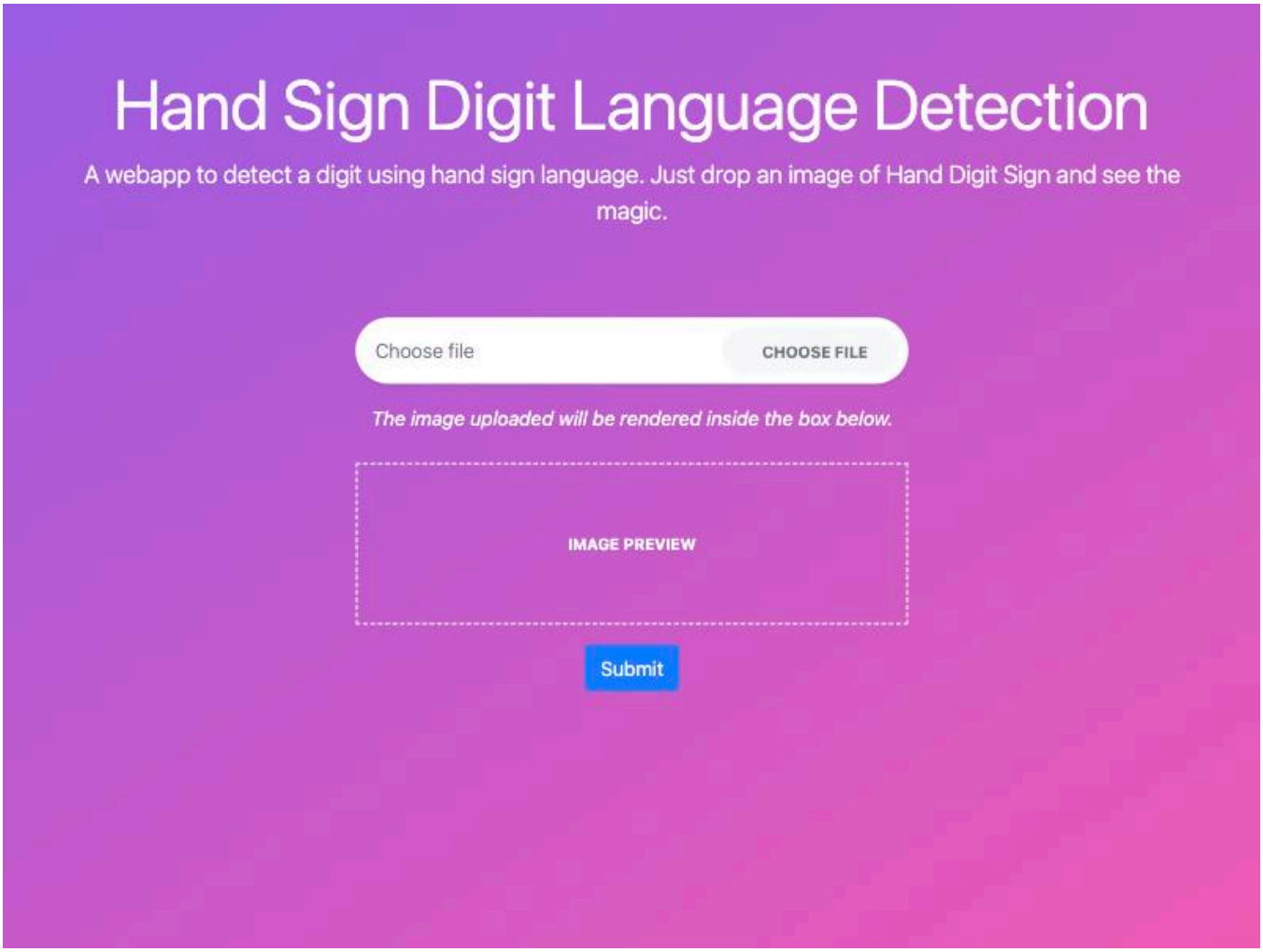


Figure 1. The home page served at `http://127.0.0.1:9000/`.

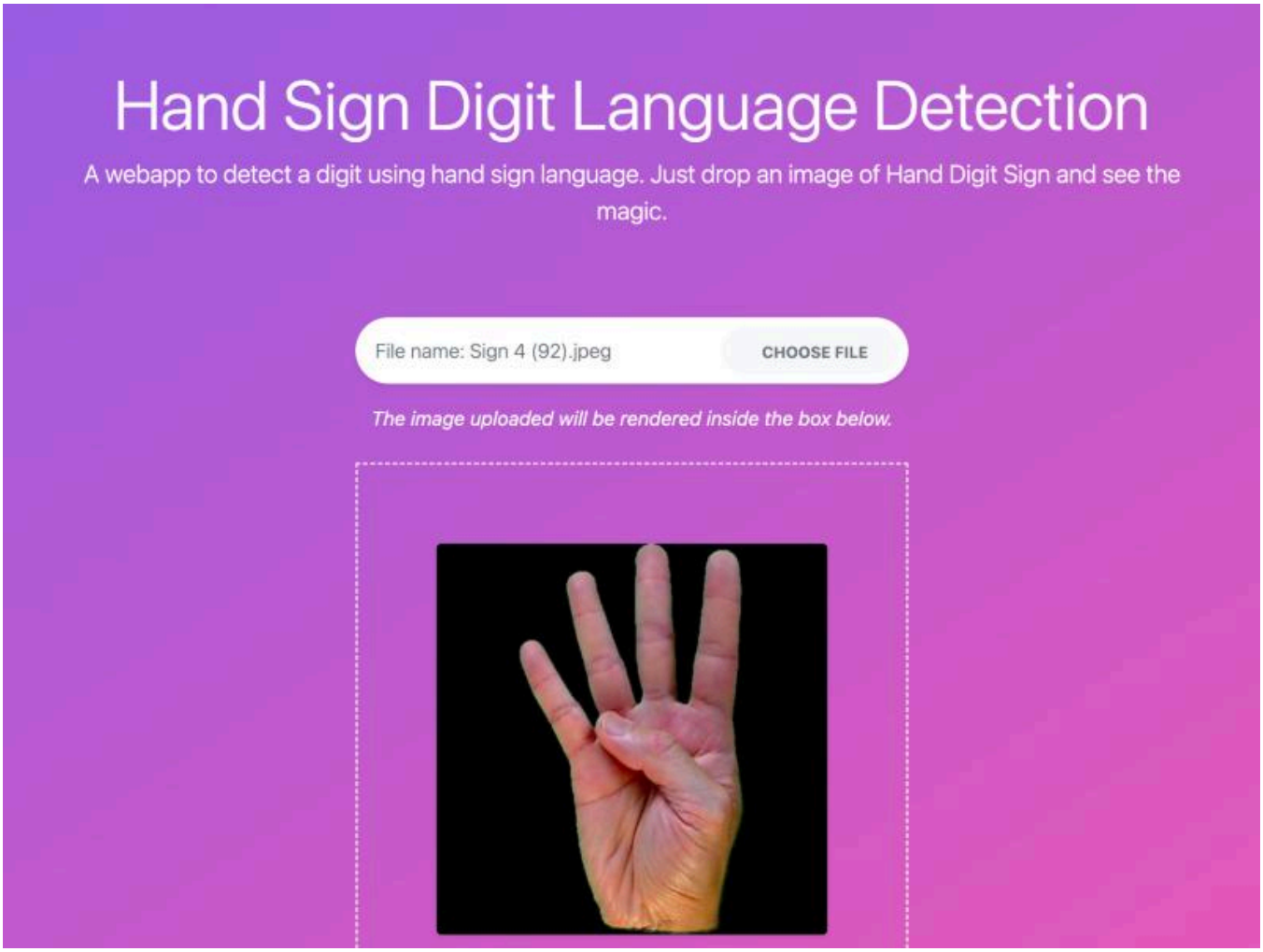


Figure 2. An image selected. `static/js/image_upload.js` previews it client-side and writes the file name into the label. Nothing has reached the server yet.



Figure 3. The prediction. A hand sign for 4 returns `4`.

Verified from the command line as well:

```
test_images/4/Sign 4 (92).jpeg -> shape (1, 224, 224, 3) pred 4
test_images/7/Sign 7 (54).jpeg -> shape (1, 224, 224, 3) pred 7
```

2. Legacy project exploration

Main files

FILE	PURPOSE
`app.py`	The Flask entry point. Creates the application and defines both routes: `/` serves the upload form, `/prediction` accepts the file and returns the digit.
`model.py`	The machine-learning layer. Loads the Keras model at import time; `preprocess_img` turns an upload into a model-ready array and `predict_result` runs inference.
`digit_model.h5`	The trained model: 28 MB, Keras 2 HDF5, ten output classes.
`templates/layout.html`	Jinja2 base template. Loads Bootstrap 4.3 and jQuery from a CDN plus the project's own CSS and JavaScript.
`templates/index.html`	The upload page: a `multipart/form-data` form with a file input named `file`.
`templates/result.html`	Shows `predictions` on success, `err` on failure.
`static/css/custom.css`	Gradient background and upload-area styling.
`static/js/image_upload.js`	Client-side only: previews the chosen image and displays its name.
`test_images/`	100 sample images in ten folders.

Where the Flask application is configured

In `app.py`:

```
app = Flask(__name__)
...
if __name__ == "__main__":
    app.run(port=9000, debug=True)
```

There is no config object, settings module or environment handling. `Flask(__name__)` takes every default, which is how `templates/` and `static/` are located. The port and debug flag are set inside the `__main__` guard, so they apply only when the file is run directly; started through the Flask CLI, the app uses the CLI's default port 5000.

Where the model is loaded

In `model.py`, at **module scope** rather than inside a function:

```
MODEL_PATH = os.path.join(os.path.dirname(__file__), "digit_model.h5")
model = load_model(MODEL_PATH)
```

This is the single most consequential detail in the codebase. Because the call sits at module level, the whole 28 MB model is read and the network built the moment anything imports `model` — including `app.py` at startup, and including any test file. It is loaded once per process and held in a module-level global that

`predict_result` closes over.

The benefit is that inference does not pay the loading cost per request. The cost is that the module cannot be imported at all without the model present, which is what forces the test suite to mock it.

Where uploaded images are preprocessed

In `model.py`, `preprocess_img`: open with Pillow, resize to the 224×224 the model expects, convert to an array scaled from 0–255 down to 0–1, reshape into a batch of one.

Two observations. The parameter is named `img_path`, but `app.py` passes a file *stream*; Pillow accepts either, so the name is misleading rather than wrong. And the resize does not preserve aspect ratio, so a non-square image is distorted rather than cropped.

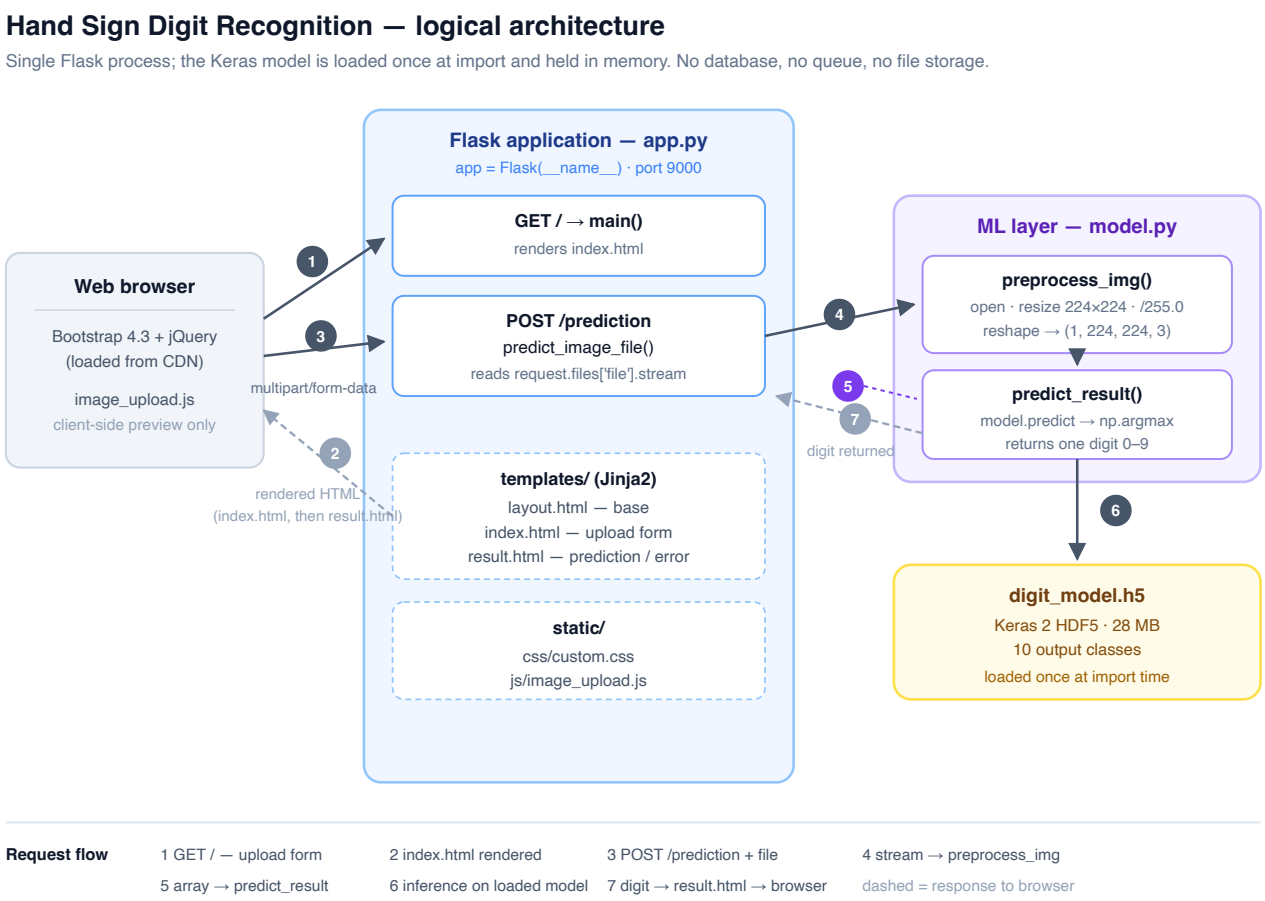
Where templates and static resources live

At the Flask defaults — `templates/` and `static/`, resolved relative to `app.py`. Bootstrap and jQuery are not vendored; they load from a CDN, so the styled pages need an internet connection.

How an upload becomes a prediction

1. The browser requests `GET /`; `main()` renders `index.html`.
2. The user picks a file. `image_upload.js` previews it locally. Nothing has been sent.
3. Submitting posts `multipart/form-data` to `POST /prediction` with the file under the field `file`.
4. `predict_image_file()` passes `request.files['file'].stream` to `preprocess_img`, which returns a `(1, 224, 224, 3)` array.
5. `predict_result` calls `model.predict`; the model returns ten class probabilities and `np.argmax` reduces them to one digit.
6. The digit is rendered into `result.html` and returned.
7. If anything in 4–6 raises, the error page is rendered instead.

Architecture diagram



One Flask process holding the model in memory, serving a browser over HTTP. No database, no queue, no separate model server, no persistence — the uploaded file is read from the request stream and never written to disk.

---

3. Pylint and pre-commit

Pylint configuration

`.pylintrc` is deliberately short. The approach was to keep Pylint's defaults and fix what it reported, rather than disable checks to raise the score.

SETTING	VALUE	REASONING
`ignored-modules`	`keras`, `tensorflow`	Both build much of their public API lazily at import time. Static analysis cannot see those attributes and reports false `no-member` errors.
`fail-under`	`9.0`	Pylint exits non-zero below this, so a regression fails the build instead of being merely reported.
`max-line-length`	`100`	Pylint 3's own default, stated explicitly rather than inherited silently.

Only `duplicate-code` is disabled outright. Exactly one message is suppressed inline, in `conftest.py`, with its reason beside it:

```
from app import app  # pylint: disable=import-outside-toplevel
```

Importing `app` at module scope would load the model before the patch that replaces the loader is active, which is the entire point of that fixture.

Findings and what was done about them

The baseline scored **7.50/10**:

```
app.py:1:0:  C0114: Missing module docstring
app.py:11:0: C0116: Missing function or method docstring
app.py:17:0: C0116: Missing function or method docstring
app.py:24:4: W0702: No exception type(s) specified (bare-except)
app.py:17:0: R1710: Either all return statements should return an expression, or none
model.py:1:0:  C0114: Missing module docstring
model.py:14:0: C0116: Missing function or method docstring
model.py:23:0: C0116: Missing function or method docstring
```

FINDING	RESOLUTION
`C0114`, `C0116` (six)	Docstrings added. `model.py`'s records that the model loads once per process; `preprocess_img`'s notes that it accepts a path *or* a stream, since the parameter name suggests otherwise.
`W0702` bare `except:`	Replaced with `except (KeyError, OSError, ValueError)` — one per failure a bad upload actually produces — plus `app.logger.exception` so the cause reaches the log. The bare form also caught `KeyboardInterrupt` and `SystemExit`, so the server could not be interrupted cleanly during a request.
`R1710` inconsistent returns	The body was wrapped in `if request.method == 'POST'`, which the route already guarantees. If it were ever false the view returned `None`, which Flask rejects. Removing the redundant check removed the path.

Final score: **10.00/10**.

A note on the score

The assignment states that a perfect score is not the objective, and the numbers bear that out. The six docstrings moved the score most — 7.50 to 9.38 — and changed no behaviour whatsoever. The two findings that actually mattered moved it least: the bare `except` was concealing the cause of every failure, and the inconsistent return left a live path on which the view returned `None`. **The score moved least for the fixes that mattered most**, which is the clearest argument against treating it as the target.

Pre-commit configuration

`.pre-commit-config.yaml`, installed with `pre-commit install`:

HOOK	PURPOSE
`trailing-whitespace`, `end-of-file-fixer`	Whitespace hygiene. The first run fixed missing end-of-file newlines in four templates and the stylesheet.
`check-yaml`	Validates the Actions workflow before it reaches CI.
`check-added-large-files` (`--maxkb=30720`)	Set just above `digit_model.h5` at 28 MB, so the model commits but nothing larger slips in.
`check-merge-conflict`	Blocks conflict markers.
`debug-statements`	Blocks a stray `breakpoint()` or `pdb` import.
`pylint` (local)	Runs Pylint from the active interpreter so it resolves Flask and Keras the way CI does.

---

4. Pytest

Nineteen tests across `tests/test_routes.py` and `tests/test_model.py`.

REQUIREMENT	TESTS
Home page accessible	`test_home_page_is_served`
Valid request → expected response	`test_valid_upload_returns_the_predicted_digit`, which also asserts the model received exactly one correctly shaped, normalised batch
Missing uploaded file	`test_missing_file_is_rejected`
Invalid or unexpected input	`test_non_image_upload_is_rejected`, `test_empty_filename_is_rejected`, `test_greyscale_image_is_rejected`, `test_get_is_not_allowed_on_the_prediction_route`
Preprocessing / prediction component	Nine tests covering shape, dtype, normalisation, path and stream inputs, and the argmax reduction

Edge cases

- **1×1 and 4000×10 images** — extreme aspect ratios still resize to 224×224.
- **An image already 224×224** — the resize is a no-op and must not corrupt anything.
- **A greyscale image** — preprocesses to one channel, so the reshape to three raises `ValueError`; the application rejects it rather than predicting from wrong data.
- **An empty filename** — what a browser sends when the form is submitted with nothing chosen.
- **A prediction that is not 4** — `test_predict_result_follows_the_probabilities` points the stub at class 9, proving the result tracks the model rather than being hard-coded.
- **The stub itself** — `test_stub_probabilities_are_a_valid_distribution` checks the fixture still resembles real model output, so the suite cannot drift into testing a nonsense stand-in.

Mocking

Because `model.py` loads the model at import, every test run would otherwise pay that cost before collection. `conftest.py` patches `keras.models.load_model` at module import — before any test module imports `app` or `model` — so the application receives a stub whose `predict` returns a fixed ten-class vector. **The suite runs in 0.44 seconds.**

One test, marked `slow`, exercises the genuine model through a saved reference to the unpatched loader, asserting a real sample of the digit 4 is classified as 4 with more than 50% confidence. It also asserts the fixture returned a real Keras object: the stub peaks at class 4 too, so without that check the test would pass whether or not the real model had loaded. A test that cannot fail is worse than no test.

Local execution

```
$ pylint app.py model.py conftest.py tests/
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.73/10, +0.27)

$ pytest
tests/test_model.py ............                                [ 63%]
tests/test_routes.py .......                                    [100%]
============================== 19 passed in 0.44s ==============================
```

Raw output is committed under `docs/evidence/`.

---

5. GitHub Actions

`.github/workflows/ci.yml` runs on every push and pull request to `main`:

1. Check out the repository
2. Set up Python 3.11 with a pip cache keyed on both requirements files
3. Install the application and development dependencies
4. Run the pre-commit hooks
5. Run Pylint
6. Run pytest

A green run completes in about 47 seconds, of which 25 is the dependency install — TensorFlow resolves far faster on Linux than on macOS, where no matching wheel exists at all.

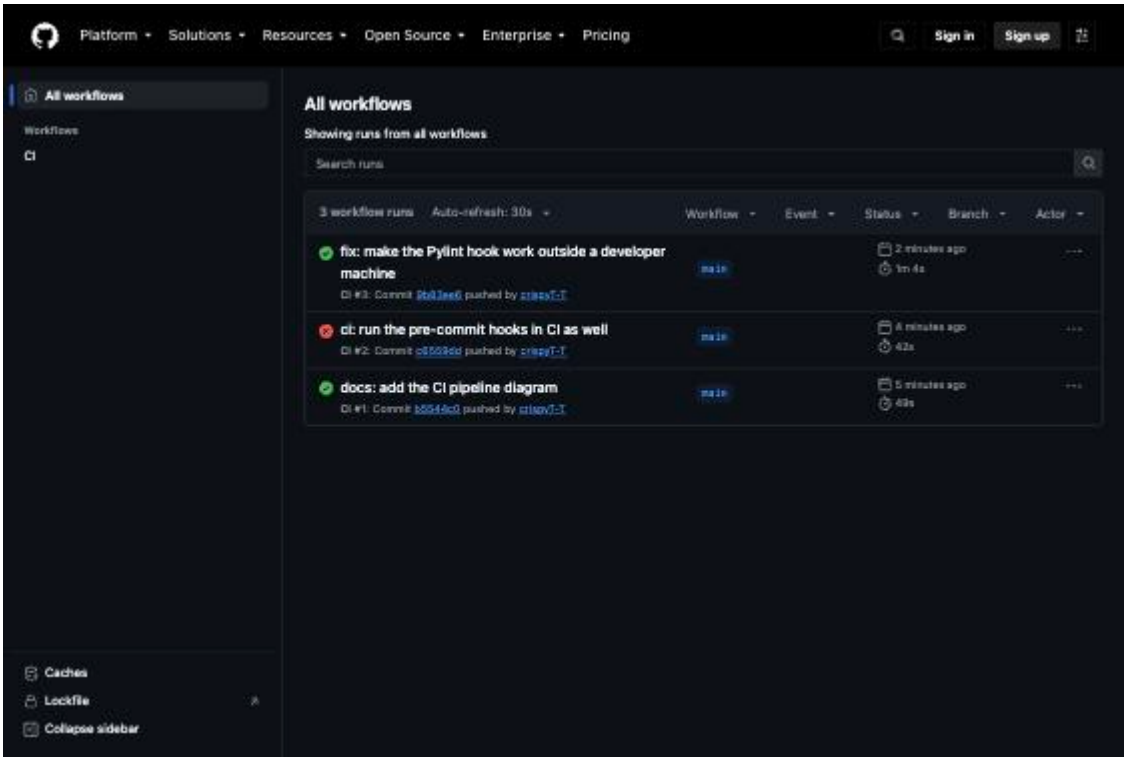
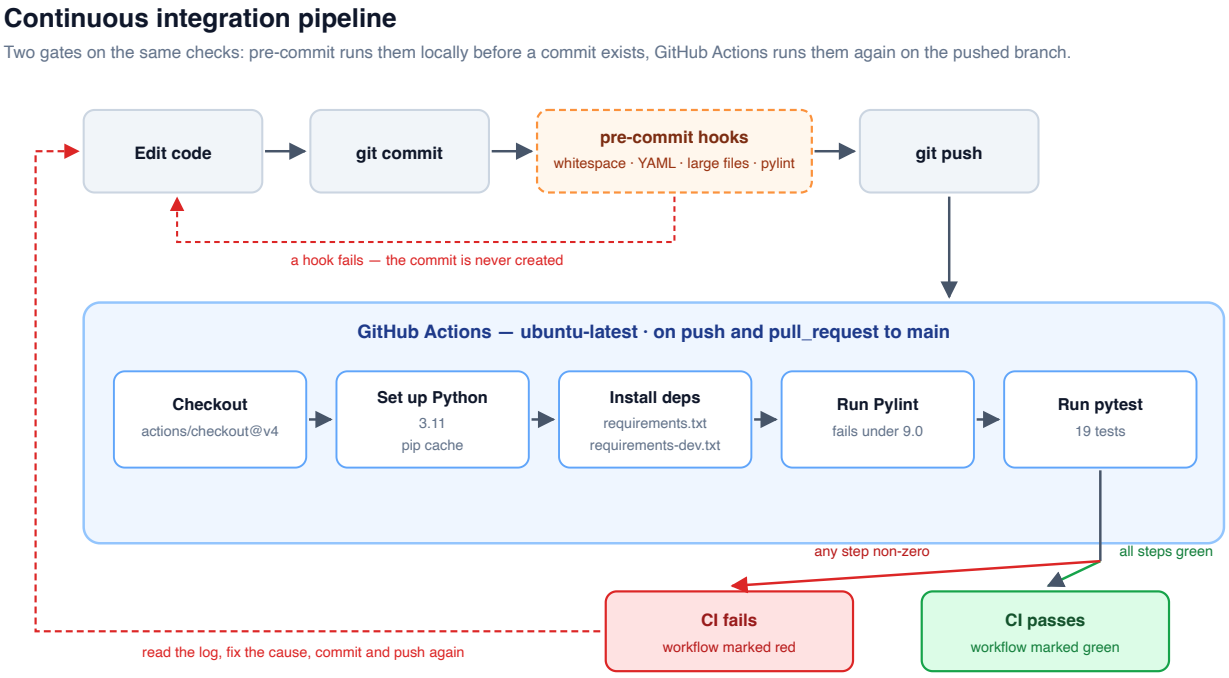


Figure 4. The workflow runs: #1 green, #2 red, #3 green.

CI pipeline diagram



The same checks run at two gates. Pre-commit runs them locally before a commit exists; Actions runs them again on the pushed branch, because local hooks can be bypassed with `--no-verify` and CI should not assume they ran.

6. CI progression

A failed check and its correction

Run **#2** failed. The commit added `pre-commit run --all-files` as a CI step, on the reasoning that CI should not trust that local hooks had run. The step failed immediately:

```
pylint...................................................................Failed
- hook id: pylint
Executable `.venv/bin/pylint` not found
```

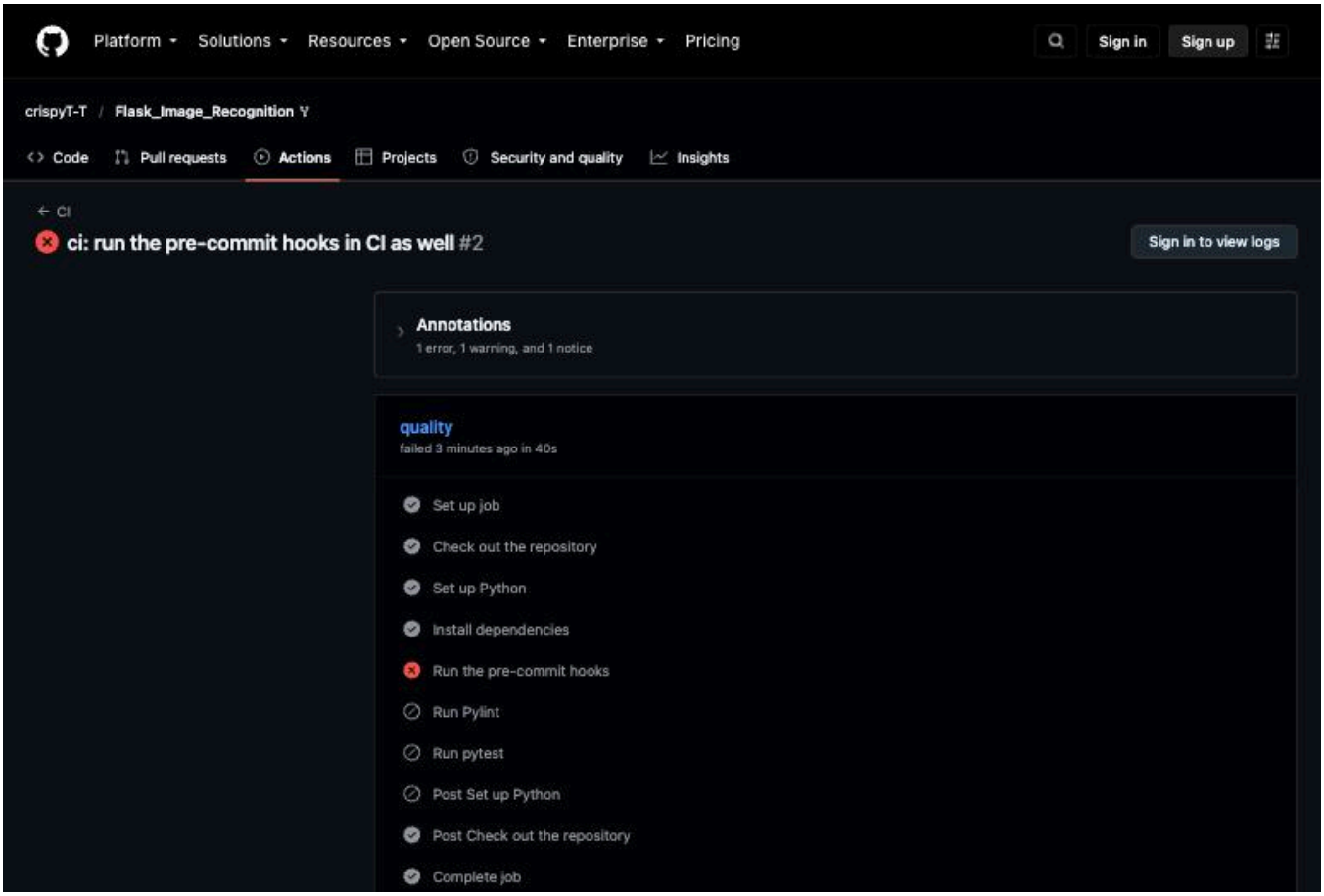


Figure 5. Run #2: the pre-commit step failed and the later steps were skipped.

The cause was a genuine defect in the pre-commit configuration. The Pylint hook was declared as:

```
entry: .venv/bin/pylint
language: system
```

That path exists only on a machine where someone created a virtualenv in that exact location. A CI runner installs dependencies into the interpreter `setup-python` provides and has no `.venv` at all. The hook had worked locally for every commit until then, and nothing would have revealed the problem except running it somewhere else — which is precisely what CI is for.

The fix was to invoke the module rather than a path:

```
entry: python -m pylint
```

This resolves to the activated virtualenv locally and to the runner's interpreter in CI, so both see Flask and Keras and resolve imports identically. Run **#3** passed.

RUN	COMMIT	RESULT
#1	`807e663` docs: add the CI pipeline diagram	✅ success
#2	`7d154b6` ci: run the pre-commit hooks in CI as well	❌ **failure**
#3	`259f2c0` fix: make the Pylint hook work outside a developer machine	✅ success

The hashes above are the current ones. The run pages on GitHub record the hashes those commits had when the runs executed, which differ: the history was rewritten afterwards to correct the author name on every commit, which changes every hash while leaving the content identical. The commit messages are the reliable way to match a run to a commit.

Quality progression in the commit history

Two further sequences were developed before the repository was pushed to GitHub, and are visible in the commit history with their recorded output in `docs/evidence/`:

A Pylint failure followed by a correction. Commit `1090844` added the Pylint configuration while the code scored 7.50/10 — below the `fail-under=9.0` threshold, so Pylint exited non-zero. `f7d1742` added the missing docstrings (9.38) and `76574b5` fixed the error handling (10.00).

A failing test followed by a fix. Writing the error-path tests exposed a real defect: both the success page and the error page were returned with **HTTP 200**, so nothing reading the status code could distinguish a rejected upload from a prediction. Four tests failed:

```
FAILED tests/test_routes.py::test_missing_file_is_rejected - assert 200 == 400
FAILED tests/test_routes.py::test_non_image_upload_is_rejected - assert 200 == 400
FAILED tests/test_routes.py::test_empty_filename_is_rejected - assert 200 == 400
FAILED tests/test_routes.py::test_greyscale_image_is_rejected - assert 200 == 400
4 failed, 15 passed in 1.43s
```

Those tests were committed **failing** in `d6510fa` to record the defect, and `754f24a` fixed it by returning `400` from the error branch. The page a person sees is unchanged — same template, same message — so the original behaviour is preserved for anyone using a browser, while a programmatic caller can now tell the difference.

In the interest of accuracy: these two sequences are evidenced by the commit history and the saved tool output, not by red Actions runs, because they were completed before the repository existed on GitHub. The CI failure in run #2 is the one caught by Actions itself.

Commit history

```
259f2c0  fix: make the Pylint hook work outside a developer machine     CI #3 pass
7d154b6  ci: run the pre-commit hooks in CI as well                     CI #2 FAIL
807e663  docs: add the CI pipeline diagram                              CI #1 pass
72a1bf0  ci: run Pylint and pytest on every push with GitHub Actions
f200431  docs: write up the Pylint, pre-commit and pytest work
754f24a  fix: return 400 when the upload cannot be processed            tests pass
d6510fa  test: cover the routes, preprocessing and prediction           4 tests FAIL
219db52  ci: run formatting checks and Pylint before every commit
76574b5  fix: catch the failures we expect and log the ones we do not   10.00/10
f7d1742  style: add the module and function docstrings Pylint asked for  9.38/10
1090844  ci: add a Pylint configuration                                  7.50/10
1d079e1  docs: describe the legacy structure, request flow and architecture
7c12c5a  docs: record the setup problems and working run
e07079b  build: pin dependencies that install on current platforms
d8c1b2c  fix: load the model from a path that exists on any machine
960985c  chore: ignore editor, cache and virtualenv files
62d8114  Added dl model                                          <- legacy baseline
```

---

7. Reflection

Three things stood out.

The legacy application could not run anywhere but its author's machine. Between the hardcoded `D:/SACHIN/...` model path and dependencies with no wheels for the current platform, neither problem was visible by reading the code — both surfaced only on trying to run it. A CI pipeline would have caught both on the first push, years earlier.

The Pylint score was the least useful signal it produced. The docstrings moved it most and mattered least; the bare `except` moved it little and mattered most. The score is a summary of how many messages remain, not of how serious they were, and optimising it directly would have meant doing the cheap work first.

CI earned its place immediately. The one genuine failure was a defect no amount of local testing could have exposed, because the defect *was* the dependence on a local machine. That the failure appeared on the very first run with the hooks enabled is a fair illustration of the point: a check that only ever runs in one environment is not really a check.